Patrol Grid Visit

Technical Guide

Patrol Coverage Analysis — Methodology & Calculation Reference

Guide version 1.1 · Workflow version 1.0.0 · Generated October 07, 2026

Author: Tevin Temu, Wildlife Dynamics

1. Overview

The **Patrol Grid Visit** workflow measures how often each part of a landscape is visited by ranger patrols. It pulls the GPS tracks of completed patrols from **EarthRanger**, cleans them into trajectory segments, lays a regular square grid over the patrolled area, and counts the number of **distinct patrols** that passed through each grid cell. The result is a colour-classified coverage map, delivered as an interactive HTML map and a single-widget dashboard.

The workflow is implemented as an **Ecoscope Workflow** (YAML spec) and executes within the ecoscope-platform task/compiler runtime. Each task in the spec is a composable, versioned function; tasks are chained via data references (${{ workflow.<id>.return }}) and the engine resolves execution order automatically.

Workflow details

Field	Value
Name	Patrol Grid Visit
Workflow version	1.0.0
Author	Tevin Temu (Wildlife Dynamics) — tevin@wildlifedynamics.com
Data source	EarthRanger (completed patrols)
Repository	github.com/wildlife-dynamics/patrol-grid-visit
License	BSD 3-Clause

Note: The map shows *how many different patrols* entered each cell, not how many GPS fixes fall inside it. A single long patrol that loops through a cell several times still counts once for that cell.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All patrol data is sourced from an **EarthRanger** instance (set_er_connection task). The user selects a pre-configured EarthRanger data source; the resulting client is passed to get_patrol_observations. Authentication is handled by the Ecoscope EarthRanger client; no credentials are stored in the spec.

2.2 Patrol Selection

get_patrol_observations fetches the track points recorded during patrols in the analysis time range. Only the **Patrol Types** field is exposed in the form; the remaining fetch options are fixed by the spec:

Parameter	Value	Effect
patrol_types	User-set (default: empty)	Patrol type value(s) to analyse. Leave empty to include all patrol types.
status	[done]	Only completed patrols are included. Active, scheduled, or cancelled patrols are ignored.
patrols_overlap_daterange	true	A patrol is included if any part of it overlaps the time range, not only if it starts inside it.
include_patrol_details	true	Patrol metadata (including the patrol ID) is attached to every observation.
raise_on_empty	true	If no patrol observations are found, the workflow stops with an error rather than producing an empty map.

Parameter	Value	Effect
sub_page_size	200	Page size used when paging through patrols in the EarthRanger API.

2.3 Base Map Tile Layers

The background map is configured by set_base_maps_pydeck. It ships with two stacked ArcGIS tile layers, both user-editable in the form. The first layer in the list is drawn at the bottom:

Order	Tile service	Opacity	Max zoom
1 (bottom)	ArcGIS World Topo Map	1.0	20
2 (top)	ArcGIS World Imagery	0.5	20

3. Data Ingestion Pipeline

3.1 Observations → Relocations

Raw patrol observations are converted into a standardised **relocations GeoDataFrame** by process_relocations. Only the following columns are retained:

Column	Description
groupby_col	Subject (tracked device / patrol leader) identifier used to build tracks
fixtime	UTC datetime of the GPS fix
junk_status	True if the fix is flagged as junk in EarthRanger
geometry	Point geometry (WGS 84)
extra__created_at	Record creation timestamp
patrol_id	Identifier of the patrol the fix belongs to

Three known-bad coordinate pairs are filtered out unconditionally:

- (180.0, 90.0) — boundary sentinel
- (0.0, 0.0) — null-island artefact
- (1.0, 1.0) — common default / test value

3.2 Trajectory Segment Filter

A **trajectory segment filter** defines which segments between consecutive fixes are kept. It is not a separate task in this workflow: the trajectory_segment_filter argument of relocations_to_trajectory is left unset in the spec, so it appears as an advanced field under the **Relocations To Trajectory** step of the form. All six thresholds are user-configurable and default to the ecoscope-platform TrajectorySegmentFilter values:

Parameter	Default	Unit	Purpose
min_length_meters	0.001	m	Remove sub-centimetre GPS noise
max_length_meters	100 000	m	Remove only extreme jumps (> 100 km)
min_time_secs	1	s	Remove zero-duration segments
max_time_secs	172 800	s	Remove gaps > 48 h (device off / signal lost)

Parameter	Default	Unit	Purpose
min_speed_kmhr	0.01	km/h	Remove near-stationary segments
max_speed_kmhr	500	km/h	Remove only physically impossible speeds

Note: These defaults are deliberately permissive so that foot, vehicle, motorbike, and aerial patrols are all retained. The trade-off is that long straight segments spanning GPS gaps (up to 100 km or 48 h) are kept and are drawn across every cell they cross, crediting those cells with a visit. For foot patrols, tighter limits such as max 5 000 m, 21 600 s, and 9 km/h give a more faithful coverage map.

3.3 Relocations → Trajectories

Relocations are joined into **linear trajectory segments** by relocations_to_trajectory. Each segment connects two consecutive fixes of the same subject; segments outside the filter limits above are then dropped. Each segment records, among other attributes:

- **dist_meters** — straight-line distance between the two fixes
- **timespan_seconds** — elapsed time between the two fixes
- **speed_kmhr** — average speed over the segment
- **geometry** — LineString in WGS 84

Subjects with only a single relocation in the period cannot form a segment and are excluded (a UserWarning is logged for each).

3.4 Column Mapping

The trajectory builder carries non-core relocation columns through with an extra__ prefix. map_columns renames extra__patrol_id back to patrol_id, which the coverage grid requires. No columns are dropped, and missing columns do not raise an error (raise_if_not_found: false).

4. Patrol Coverage Grid — Methodology

The core calculation is performed by create_patrol_coverage_grid (package ecoscope-workflows-ext-mnc).

4.1 Grid Construction

A regular square mesh is generated over the extent of the trajectories by create_meshgrid, using the user-set **Grid Cell Size** (default **1 000**) as the cell edge length. Only cells that intersect a trajectory are generated (intersecting_only: true). The grid is built in **EPSG:3857 (Web Mercator)**, and each cell receives a grid_id.

Note: Cell size is measured in EPSG:3857 units. Web Mercator stretches distances by a factor of 1 / cos(latitude): near the equator (e.g. East Africa, within ±5°) this is under 0.4 %, so a 1 000 unit cell is effectively 1 km. At 20° from the equator, cells are about 6 % smaller on the ground than the nominal size, and about 15 % smaller at 30°.

4.2 Segment Clipping & Time Allocation

Trajectory segments are reprojected to the grid CRS; segments with zero length are discarded. The remaining segments are intersected with the grid (geopandas.overlay, how="intersection"), splitting each segment into the pieces that fall inside each cell. For each piece:

- **clipped_dist_meters** = length of the piece inside the cell
- **clipped_timespan_seconds** = segment timespan × (clipped length ÷ segment length) — time is allocated in proportion to distance, assuming constant speed along the segment

4.3 Per-Cell Metrics

The clipped pieces are grouped by grid_id and aggregated:

Column	Calculation	Used on map
unique_patrol_count	Number of distinct patrol_id values with at least one piece in the cell	Yes — classified and coloured
time_spent_seconds	Sum of clipped_timespan_seconds	No
time_spent_hours	time_spent_seconds ÷ 3 600, rounded to 2 dp	No
distance_patrolled_meters	Sum of clipped_dist_meters	No
distance_patrolled_km	distance_patrolled_meters ÷ 1 000, rounded to 2 dp	No

The result is sorted by unique_patrol_count, highest first.

4.4 Empty Cells and Area of Interest

The spec fixes keep_empty_cells: false, so cells with no patrol activity are dropped and the map only shows visited cells. It also fixes aoi: null, so the grid follows the trajectory extent rather than a protected-area boundary. Neither option is exposed in the form; changing them requires editing spec.yaml and recompiling.

5. Classification & Styling

5.1 Classification

apply_classification bins unique_patrol_count into **5 classes** using the **Equal Interval** scheme: the range between the least- and most-visited cells is divided into five bins of equal width. Bin labels are written as ranges with 0 decimal places (for example 1 - 4) into the patrol_count_bins column.

> **Note:** Equal interval is sensitive to outliers. If one cell (e.g. a ranger post) is visited by far more patrols than anywhere else, most cells will fall into the lowest bin. In that case, consider switching the scheme in spec.yaml to quantile or natural_breaks.

5.2 Colour Map

apply_color_map assigns each bin a colour from the matplotlib **RdYlGn** (red–yellow–green) colormap and stores it in bin_colors.

5.3 Grid Layer Style

Property	Value
Fill	bin_colors, opacity 65 %
Outline	Black, 0.75 px (pixel units)
Pickable	Yes (enables hover interaction)
Legend	Title “Grid visits”, labels from patrol_count_bins, bottom-right

6. Map & Dashboard

- **View state** — compute_view_state_from_gdf fits the initial camera to the extent of the classified grid (pitch 0, bearing 0, max zoom 18).

- **Map** — draw_map (package ecoscope-workflows-ext-big-life) renders the grid layer over the base map tiles as an interactive DeckGL map (max zoom 20).
- **Persist** — persist_text writes the map HTML to patrol_grid_visits.html in the results directory.
- **Dashboard** — create_map_widget_single_view wraps the map in a widget titled **Patrol Grid Visits**, and gather_dashboard assembles the dashboard with the workflow details and time range.

7. Output Files

All outputs are written to the directory defined by the ECOSCOPE_WORKFLOWS_RESULTS environment variable.

File	Format	Content
patrol_grid_visits.html	HTML	Interactive patrol grid visit map
Dashboard	Runner UI	Single-widget dashboard containing the map

8. Workflow Execution Logic

8.1 Grouping

Groupers are set to an empty list, so the workflow produces **one map** covering all selected patrols. There are no per-subject or per-period views.

8.2 Skip Conditions

Every task inherits two default skip conditions:

- **any_is_empty_df** — if any input DataFrame is empty, the task is skipped rather than raising an error.
- **any_dependency_skipped** — if an upstream task was skipped, all tasks that depend on it are skipped too.

> **Note:** Because get_patrol_observations uses raise_on_empty: true, an empty patrol query fails the run with an error at the first data step instead of silently producing an empty dashboard.

9. Software Versions

The workflow's requirements, as pinned in spec.yaml:

Package	Version	Role
ecoscope-platform	2.18.0	Core task library and workflow engine
ecoscope-workflows-ext-mnc	1.0.5	Patrol coverage grid task
ecoscope-workflows-ext-big-life	1.0.2.*	draw_map task
ecoscope-workflows-ext-custom	0.1.0rc14.*	create_geojson_layer task
ecoscope-workflows-ext-ste	0.0.0rc1.*	STE utility tasks
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	Shared custom tasks
ecoscope-workflows-ext-mep	1.0.4	Shared custom tasks
pydeck	0.9.2	Renders the interactive DeckGL map

Package	Version	Role
opentelemetry-sdk	>=1.20.0, <2.0.0	Workflow run telemetry

ecoscope-platform is pulled from the ecoscope-workflows prefix.dev channel; the ecoscope-workflows-ext-* packages from the ecoscope-workflows-custom channel; pydeck and opentelemetry-sdk from conda-forge. The runtime environment is managed by **pixi**.